

토스페이먼츠 테스트 결제 연동 보고서 — 프리미엄 아카이브

1. 한눈에 보기

- 토스페이먼츠는 **테스트 모드**를 제공한다. 사업자등록 없이, 실제 카드번호를 넣어도 돈이 빠져나가지 않는 가상 결제 환경이다. 우리 CLAUDE.md의 "결제는 테스트 모드로만" 규칙과 정확히 맞는다.
- 흐름은 **두 단계**다. ① 브라우저에서 결제창을 띄워 카드 인증 → ② 우리 **서버**가 토스 API에 "이 결제 승인해줘"라고 확인 요청. ②를 브라우저에서 하면 금액 위변조로 공짜 멤버십이 뚫린다.
- 키는 두 개. **클라이언트 키**(공개해도 됨, `NEXT_PUBLIC_` 붙임)와 **시크릿 키**(절대 노출 금지, `NEXT_PUBLIC_` 붙이면 안 됨).
- 우리 MVP는 **정기 구독(빌링)이 아니라 1회 결제**로 간다. 토스는 정기결제 스케줄링을 대신 돌려주지 않아서 직접 만들어야 하고, PRD "이번 단계에서 하지 않는 것"에 구독 해지 화면도 빠져 있다.
- 결제 승인이 성공한 **그 서버 코드 안에서** 회원의 멤버십 상태를 켜다. 그 지점이 PRD 5번 흐름("결제가 끝나면 프리미엄 글을 읽을 수 있는 상태")의 유일한 스위치다.

2. 사전 준비 (계정 / 키 발급)

용어 한 줄 풀이

- API 키**: 토스 서버에 "나 어느 상점이야"를 알려주는 비밀번호 같은 문자열.
- 테스트 모드**: 실제 돈이 오가지 않는 연습용 환경. 라이브 환경과 완전히 분리돼 있다.

2-1. 두 가지 시작 방법

방법	절차	언제 쓰나
A. 문서용 테스트 키	가입 없이 공식 문서에 공개된 테스트 키를 그대로 복사해 사용	오늘 당장 흐름만 확인
B. 개발자센터 가입	이메일·휴대폰만으로 가입 → "개발 체험 상점" 생성 → 내 테스트 키 발급	API 로그·웹훅까지 보고 싶을 때 (권장)

가입 주소: <https://developers.tosspayments.com> → 로그인 후 **내 개발정보 > API 키**에서 키 확인. 사업자 등록번호는 테스트 단계에서 필요 없다 (실제 결제 전환 시에만 전자결제 심사 필요 — 우리는 안 함).

확인 필요: 공식 문서에 공개된 "문서용 테스트 키"의 정확한 문자열은 이번 조사에서 원문 값을 확인하지 못했다. 문서 페이지(아래 참고 링크의 [how-to-test-toss-payments](#))에서 직접 복사하거나, 방법 B로 본인 키를 발급받는 편이 안전하다.

2-2. 클라이언트 키 vs 시크릿 키

	클라이언트 키	시크릿 키
하는 일	브라우저에서 결제창을 띄움	서버에서 결제를 승인함
노출돼도 되나	된다 (공개 전제로 설계됨)	절대 안 된다
Next.js 환경변수 이름	NEXT_PUBLIC_TOSS_CLIENT_KEY	TOSS_SECRET_KEY (접두사 없음)
테스트 키 접두사	test_ck_ 또는 test_gck_	test_sk_ 또는 test_gsk_

접두사가 두 종류인 이유: 결제위젯(주문서형/결제창형)은 `gck` / `gsk`, 개별 연동 API는 `ck` / `sk` 를 쓴다. **클라이언트 키와 시크릿 키는 반드시 같은 짝이어야 한다.** 짝이 안 맞으면 승인 단계에서 `UNAUTHORIZED_KEY` 에러가 난다.

시크릿 키가 유출되면 개발자센터에서 재발급할 수 있고, 기존 키는 7일 뒤 만료된다.

3. 단계별 연동 방법

1단계 — SDK 설치

무엇을: 결제창을 띄우는 공식 자바스크립트 라이브러리를 설치한다. **왜:** 카드번호 같은 민감 정보를 우리 화면이 직접 받지 않기 위해서다. 토스가 제공하는 창이 대신 받는다.

```
npm install @tosspayments/tosspayments-sdk
```

CLAUDE.md "새 라이브러리를 설치하기 전에 먼저 물어본다" — 이 설치 하나는 사용자 승인을 받고 진행할 것. (패키지 최신 버전 번호는 **확인 필요**, npm 페이지가 조사 시점에 403이었다.)

2단계 — 환경변수 넣기

무엇을: 키를 코드가 아니라 `.env.local` 파일에 둔다. **왜:** CLAUDE.md 보안 규칙 — 비밀 값은 코드에 직접 쓰지 않고 깃에 올리지 않는다.

```
# .env.local (.gitignore에 반드시 포함)
NEXT_PUBLIC_TOSS_CLIENT_KEY=test_ck_...
TOSS_SECRET_KEY=test_sk_...
```

핵심 규칙 한 줄: Next.js는 `NEXT_PUBLIC_` 으로 시작하는 환경변수만 브라우저로 내려보낸다. 시크릿 키에 이 접두사를 붙이는 순간 전 세계에 공개된다. 절대 붙이지 말 것.

3단계 — 결제창 호출 (멤버십 안내 화면)

무엇을: 결제 버튼을 누르면 토스 결제창이 뜬다. **왜:** 여기서는 "인증"만 일어난다. 아직 돈은 승인되지 않은 상태다.

`app/membership/page.tsx` 안의 버튼 (클라이언트 컴포넌트, 파일 맨 위 `"use client"`)

```
import { loadTossPayments } from "@tosspayments/tosspayments-sdk";

async function 결제하기(userId: string) {
  const toss = await loadTossPayments(process.env.NEXT_PUBLIC_TOSS_CLIENT_KEY!);
  const payment = toss.payment({ customerKey: userId }); // 비회원이면 "ANONYMOUS"

  await payment.requestPayment({
    method: "CARD",
    amount: { currency: "KRW", value: 9900 },
    orderId: crypto.randomUUID(), // 6~64자, 주문마다 고유해야 함
    orderName: "프리미엄 아카이브 멤버십",
    successUrl: `${window.location.origin}/membership/success`,
    failUrl: `${window.location.origin}/membership/fail`,
  });
}
```

- `customerKey` : 이 고객이 누구인지 구분하는 값. 충분히 무작위적인 고유 값이어야 하므로 **이메일이나 1,2,3 같은 순번은 쓰지 말 것**. 회원 UUID가 적당하다.
- `orderId` : 우리가 만드는 주문번호. 6자 이상 64자 이하.
- `successUrl` / `failUrl` : 결제창이 끝난 뒤 사용자를 되돌려보낼 우리 주소.

폴더 규칙 충돌 메모: CLAUDE.md는 "화면 하나는 폴더 하나, PRD의 다섯 화면 = 폴더 다섯 개"라고 했다. 성공/실패 처리는 새 화면이 아니라 멤버십 결제의 뒷처리이므로, 위처럼 **멤버십 안내 폴더 하위** (`/membership/success` , `/membership/fail`)에 두는 편이 규칙을 덜 깬다. 최종 배치는 사용자 확인을 받는 게 좋다.

4단계 — 성공 리다이렉트 받기

무엇을: 결제창이 끝나면 토스가 `successUrl` 뒤에 결과값을 붙여서 사용자를 돌려보낸다. **왜:** 이 값들이 승인 요청에 필요한 재료다.

`successUrl` 에 붙어 오는 쿼리 파라미터: `paymentKey` , `orderId` , `amount` `failUrl` 에 붙어 오는 값: `code` , `message` , `orderId`

여기서 끝내면 안 된다. 이 화면에 도착한 것만으로는 결제가 완료된 게 아니다.

5단계 — 서버에서 결제 승인 (가장 중요)

무엇을: 우리 서버가 토스 API에 최종 승인을 요청한다. **왜:** 실제로 돈이 빠져나가는 건 이 호출이 성공했을 때다. 그리고 이 호출은 **시크릿 키**가 필요해서 서버에서만 할 수 있다.

`app/membership/success/page.tsx` (서버 컴포넌트 — 브라우저가 아니라 서버에서 실행됨)

```
export default async function Page({ searchParams }) {
  const { paymentKey, orderId, amount } = await searchParams;

  // ① 우리 DB에 저장해둔 이 orderId의 원래 금액과 비교 - 위변조 방지
  //   (주문 생성 시 orderId와 금액을 미리 저장해두어야 이 비교가 가능하다)

  // ② 토스에 승인 요청
  const res = await fetch("https://api.tosspayments.com/v1/payments/confirm", {
    method: "POST",
    headers: {
      Authorization:
        "Basic " + Buffer.from(process.env.TOSS_SECRET_KEY + ":").toString("base64"),
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ paymentKey, orderId, amount: Number(amount) }),
  });

  if (!res.ok) {
    const err = await res.json();
    return <p>결제에 실패했어요: {err.message}</p>;
  }

  // ③ 여기서만 멤버십을 켜다
  await 멤버십_켜기(로그인한_회원_id);
  return <p>멤버십이 시작되었어요.</p>;
}
```

인증 헤더 만드는 법(문서 원문 기준): **시크릿 키 뒤에 콜론 : 을 붙이고 base64로 인코딩한 뒤**
`Authorization: Basic <결과>` 로 보낸다. 콜론을 빠뜨리는 게 가장 흔한 실수다.

제한 시간: 결제 인증이 유효한 **10분 안에** 승인 API를 호출해야 한다. 넘기면 결제가 만료(**EXPIRED**)되어 승인할 수 없다.

6단계 — 왜 승인은 반드시 서버여야 하나

브라우저 코드는 사용자가 마음대로 고칠 수 있다. 승인을 브라우저에서 하면 이런 사고가 난다.

1. **금액 조작:** 9,900원 주문을 `amount=100` 으로 바꿔 승인 요청 → 100원 내고 멤버십 획득.
2. **시크릿 키 유출:** 승인에는 시크릿 키가 필요한데, 브라우저 코드에 넣는 순간 누구나 개발자도구에서 꺼내 본다. 그 키로 남의 결제를 취소하거나 조회할 수 있다.
3. **승인 없이 멤버십 개방:** 성공 화면 주소(`/membership/success`)를 그냥 브라우저에 직접 쳐서 들어가면 멤버십이 켜지는 구조가 되기 쉽다. 결제를 한 푼도 안 하고 프리미엄 글이 열린다.

그래서 토스 문서도 요청과 승인을 **일부러 분리**해 두었다고 설명한다. 서버가 "요청할 때 금액과 승인할 때 금액이 같은지" 확인해 데이터 무결성을 지키는 구조다.

우리 CLAUDE.md의 "로그인 여부와 멤버십 여부는 서버에서 확인한다"와 "프리미엄 본문은 권한 없는 사람에게 아예 보내지 않는다"가 바로 이 지점이다.

7단계 — 테스트 수단으로 검증

- **테스트용 가상 카드번호는 따로 없다.** 본인이 실제로 발급받은 카드번호를 넣어도 테스트 환경에서는 돈이 빠져나가지 않는다.
- 휴대폰 본인인증 번호는 테스트 환경에서 `000000` 을 입력한다.
- 테스트 환경 기타 특징: 가상계좌 번호는 앞에 `x` 가 붙고 실제 입금 불가(개발자센터에서 수동 입금 처리 가능), 영수증 URL은 나오지만 내용이 비어 있음, **API당 분당 100회 요청 제한**, 정산 내역 조회는 라이브에서만 가능.
- **에러 재현:** `TossPayments-Test-Code` 요청 헤더를 쓰면 라이브에서 나는 에러를 테스트 환경에서 똑같이 재현할 수 있다. 카드사 거절 같은 실패 경로를 검증할 때 유용하다.

검증 체크리스트 (완성 기준 대응)

1. 로그인 → 멤버십 안내 → 결제 버튼 → 결제창 뜸
 2. 카드 정보 입력 → 성공 화면 도착, 서버 로그에 승인 200 OK
 3. 개발자센터에서 결제 건이 보임
 4. 아까 잠겨 있던 프리미엄 글이 전문으로 열림
 5. **일부러 실패시키기:** `/membership/success?paymentKey=아무값&orderId=아무값&amount=100` 으로 직접 접속 → 멤버십이 켜지면 안 된다
-

4. 정기 구독(빌링) vs 1회 결제 — 우리는 무엇을 택하나

두 방식 비교

	1회 결제	정기결제(빌링)
호출 메서드	<code>payment.requestPayment()</code>	<code>payment.requestBillingAuth()</code>
흐름	결제창 → 승인 API 1회	카드 등록창 → <code>authKey</code> 받음 → 빌링키 발급 API(<code>POST /v1/billing/authorizations/issue</code>) → 매달 <code>POST /v1/billing/{billingKey}</code> 호출
추가로 만들어야 할 것	없음	자동 결제 스케줄러, 빌링키 안전 보관, 결제 실패 재시도, 해지 화면
토스가 대신 해주나	—	아니다. 문서 원문: "토스페이먼트에서는 자체적으로 스케줄링 기능을 제공하지 않아요"

판단: MVP는 1회 결제로 간다

근거 세 가지.

- PRD가 정기 결제를 요구하지 않는다.** PRD 4번 화면은 "결제가 끝나면 그 회원은 프리미엄 글을 읽을 수 있는 상태가 된다"이고, 5번 화면의 멤버십 상태는 "무료 회원 / 멤버십 회원" 두 값뿐이다. 다음 결제일, 남은 기간, 갱신 같은 개념이 아예 없다.
- 해지 화면이 범위 밖이다.** PRD "이번 단계에서 하지 않는 것"에 **구독 해지 화면**이 명시돼 있다. 해지할 방법 없이 매달 자동으로 돈을 빼가는 구조를 만드는 건 (테스트 모드라 해도) 잘못된 설계다.
- 빌링은 부수 작업이 크다.** 빌링키 보관, 매달 돌릴 스케줄러, 결제 실패 처리는 모두 PRD에 없는 기능이다. CLAUDE.md "시키지 않은 기능을 덤으로 붙이지 않는다"에 정면으로 걸린다.

즉 DB에는 "멤버십 회원인가" 한 가지 정보면 충분하다. 기간 만료나 갱신 개념은 넣지 않는다.

5. 우리 PRD에 맞춘 적용 포인트

멤버십이 열리는 단 하나의 지점

```
결제창 인증 성공
↓
/membership/success 도착 (아직 아무 권한 없음)
↓
서버: 금액 대조 → 토스 승인 API 200 OK ← ★ 오직 여기
↓
DB: 이 회원의 멤버십 기록 생성
↓
글 상세 서버 컴포넌트가 DB를 보고 전문/맞보기 결정
```

권한이 켜지는 곳은 이 한 군데뿐이어야 한다. 다른 어떤 경로(성공 화면 직접 방문, 브라우저 JS, 쿼리 파라미터)로도 켜지면 안 된다.

화면별 영향

PRD 화면	변경
1. 홈	변경 없음. 자물쇠는 글의 속성일 뿐 결제와 무관
2. 글 상세	서버에서 로그인 여부 + 멤버십 여부 확인. 권한 없으면 맞보기만 DB에서 꺼내 보낸다. 전문을 보내놓고 CSS로 가리면 안 된다 (CLAUDE.md 보안 규칙)
3. 로그인·회원가입	변경 없음
4. 멤버십 안내	결제 버튼 + 결제창 호출. 이미 회원이면 결제창을 아예 띄우지 않고 "이미 이용 중입니다"
5. 내 서재	DB의 멤버십 값을 그대로 보여준다

데이터 저장

- 회원의 멤버십 여부를 기록할 자리 하나.
- 결제 기록(주문번호, 금액, paymentKey, 승인 시각)은 **주문 생성 시점에 미리 저장**해야 한다. 승인 단계에서 "이 orderId의 원래 금액은 얼마였나"를 대조하려면 저장해둔 값이 있어야 하기 때문이다. 이게 금액 조작 방어의 핵심이다.

확인 필요: PRD/CLAUDE.md 어디에도 DB나 인증 라이브러리가 지정돼 있지 않다. 위 저장 구조는 어떤 DB를 쓰든 동일하게 필요한 최소 요건이며, 실제 스키마는 스택이 정해진 뒤 잡아야 한다. (→ 함께 진행한 데이터베이스 설계안 보고서 참고)

6. 주의사항 / 흔한 실수

보안 사고 3종

실수	결과	방지
시크릿 키에 <code>NEXT_PUBLIC_</code> 붙임	키가 브라우저에 그대로 노출	접두사는 클라이언트 키에만
<code>.env.local</code> 을 깃에 커밋	GitHub에 키 공개	<code>.gitignore</code> 확인, 이미 올렸다면 개발자센터에서 키 재발급 (파일만 지워도 깃 히스토리에는 남는다)
성공 화면 도착만으로 멤버십 켜기	주소만 치면 공짜 멤버십	승인 API 200 OK를 받은 뒤에만 켜기

자주 만나는 에러 코드

에러	원인	해결
<code>UNAUTHORIZED_KEY</code>	키를 잘못 넣음	클라이언트/시크릿 키가 같은 짝인지, 시크릿 키 뒤에 <code>:</code> 붙이고 base64 했는지 확인
<code>NOT_FOUND_PAYMENT_SESSION</code>	승인 요청에 문제	10분 안에 승인했는지, <code>paymentKey</code> 가 요청 때와 같은지, 키 짝이 맞는지
<code>PAY_PROCESS_ABORTED</code>	상점 계약 또는 결제사 문제	테스트/라이브 키가 섞이지 않았는지, 해당 결제수단이 이 환경에서 쓸 수 있는지
<code>REJECT_CARD_COMPANY</code>	카드사 거절	에러 메시지의 구체적 사유(한도 초과, 비밀번호 오류 등)를 사용자에게 안내

그 밖의 함정

- 10분 제한:** 성공 화면에서 승인 호출을 미루면(예: 사용자가 버튼을 한 번 더 눌러야 승인) 만료될 수 있다. 성공 화면 서버 코드에서 **즉시** 호출한다.

- `customerKey` 에 개인정보 금지: 이메일이나 전화번호를 넣지 말고 무작위 고유 값(UUID)을 쓴다.
- `orderId` 재사용 금지: 결제마다 새로 만든다. 6~64자.
- 금액 대조를 건너뛰기: 리다이렉트로 온 `amount` 를 그대로 믿고 승인하면 조작된 금액으로 승인될 수 있다. 반드시 우리 DB 값과 비교한다.
- 결제창은 클라이언트, 승인은 서버: Next.js App Router에서 결제창 호출 파일 맨 위에 `"use client"` 가 없으면 SDK가 동작하지 않는다. 반대로 승인 코드에 `"use client"` 가 있으면 시크릿 키가 노출된다. 이 경계를 헛갈리는 게 가장 흔한 초보 실수다.
- 테스트 환경 분당 100회 제한: 자동 반복 테스트를 돌릴 때 걸릴 수 있다.

조사에서 확인하지 못한 항목 (확인 필요)

- 문서용 공개 테스트 키의 정확한 문자열 값
- `@tosspayments/tosspayments-sdk` 최신 버전 번호 (npm 페이지 접근 불가)
- `requestPayment()` 의 전체 선택 파라미터 목록 (위에 적은 것은 문서에서 확인된 핵심 파라미터만)
- 승인 API의 `Idempotency-Key` 헤더 지원 여부 — 문서에서는 취소 API 맥락으로 언급되어 있었고, 승인 API에서의 지원 여부는 원문에서 확정하지 못했다
- 웹훅(결제 결과를 토스가 우리 서버에 알려주는 방식) 설정 상세 — 1회 결제 MVP에는 필수가 아니라 조사 범위에서 제외했다

7. 참고 링크

- 결제 흐름 이해하기 — 인증/승인 2단계와 서버 승인이 필요한 이유
- 환경 설정하기 (테스트 환경) — 테스트 환경 제약, 본인인증 `000000` , 분당 100회 제한
- 회원가입·사업자번호 없이 결제 테스트하기 — 문서용 테스트 키
- API 키 — 클라이언트/시크릿 키, 접두사, 재발급
- 카드/간편결제 통합결제창 연동하기 — 1회 결제 연동 (우리가 쓸 방식)
- 주문서형 결제 연동하기 — 위젯 방식
- 토스페이먼트 JavaScript SDK — SDK 메서드 레퍼런스
- 코어 API 레퍼런스 — `POST /v1/payments/confirm` 등
- 자동결제(빌링) 결제창 연동하기 — 이번엔 안 쓰지만 비교용
- 자주 묻는 질문 — 에러 코드별 원인/해결
- 배포 체크리스트
- 개발자센터 API 키 발급

요약 — 핵심 판단은 **1회 결제 채택**(빌링은 해지 화면이 PRD 범위 밖이고 스케줄러를 직접 만들어야 함)과 **승인 API 성공 지점이 멤버십을 켜는 유일한 스위치**라는 것. 실제 구현 시 [@tosspayments/tosspayments-sdk](https://github.com/tosspayments/tosspayments-sdk) 설치하는 CLAUDE.md 규칙에 따라 사용자 승인이 필요하다.